Assembly Project: Columns

Erzheng Zhang

March 9, 2026

1 Instruction and Summary

1. Which milestones were implemented?

   All 5 milestones have been implemented.

   Easy Features:

   (a) 1, gravity

   (b) 2, speed of gravity increases

   (c) 3, difficulty selection

   (d) 4, game over and retry

   (e) 6, paused

   (f) 7, ghost piece

   Hard Features:

   (a) 1, display score, calculations explained below

   (b) 4, menu for level selection

   (c) 5, Columns theme music

2. How to view the game:

   (a) Unit width in pixels: 8

   (b) Unit height in pixels: 8

   (c) Display width in pixels: 256

   (d) Display height in pixels: 256

   (e) Base Address for Display: `0x10008000` (`$gp`)

3. Game Summary:

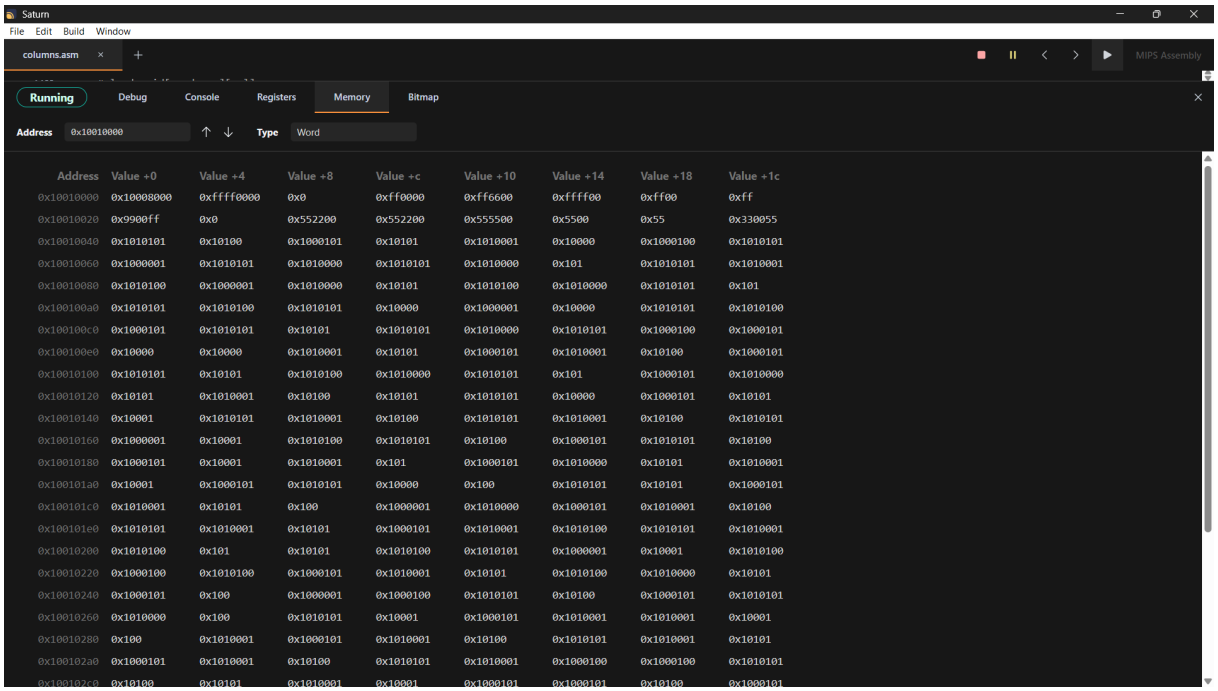
   - A falling-piece puzzle game based on the classic Sega arcade game *Columns*.
   - A column of 3 colored gems falls from the top: move left/right with `a`/`d`, shuffle gem order with `w`, and hard drop with `s`.
   - Matching 3 or more gems of the same color in a row, column, or diagonal clears them and scores points.
   - Cleared gems trigger gravity, which can cause chain reactions (combos) for multiplied scoring.
   - Three difficulty levels (Easy, Medium, Hard) affect starting gravity speed and score multiplier.
   - Falling speed increases automatically as score grows, keeping the game challenging.
   - A ghost piece previews where the column will land.
   - Press `p` to pause, `q` to quit, and `r` to retry after game over.

- Background music plays throughout using async MIDI via `syscall 31`.

Memory Layout

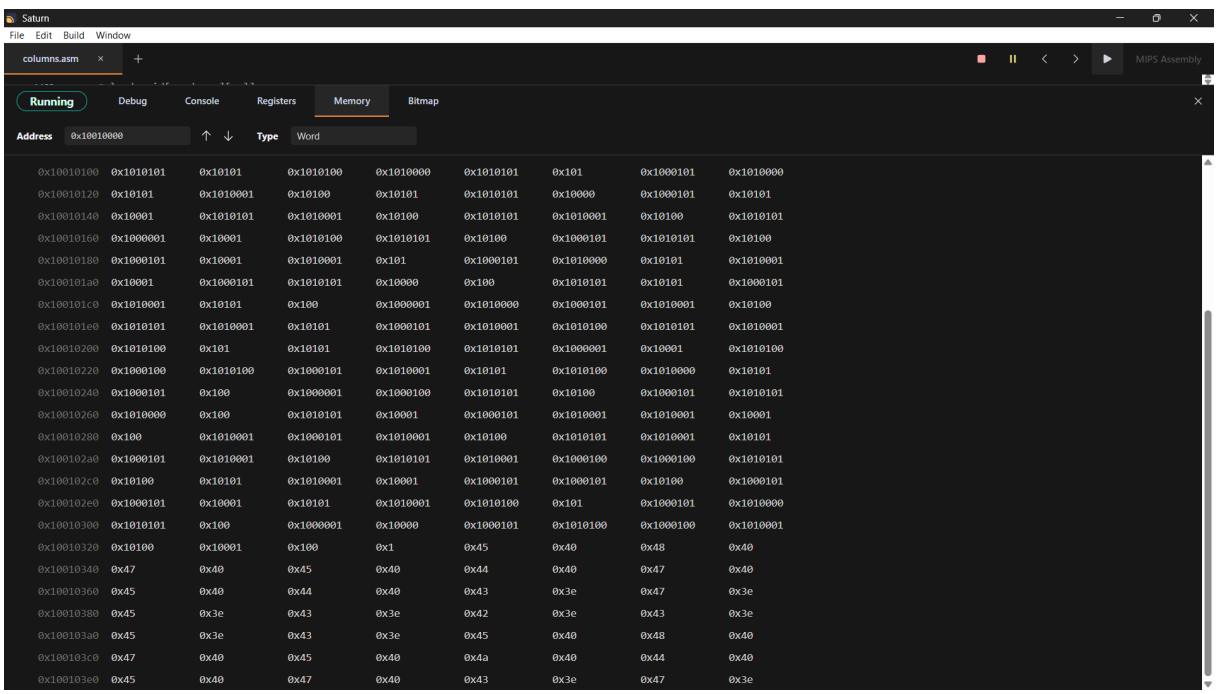
Label	Type	Size	Description
Immutable Data			
`ADDR_DSPL`	`.word`	4 B	Base address of bitmap display (`0x10008000`)
`ADDR_KBRD`	`.word`	4 B	Base address of keyboard (`0xffff0000`)
`color_table`	`.word`	28 B	7 RGB colors for gems (indices 0–6)
`ghost_color_table`	`.word`	28 B	7 dimmed RGB colors for ghost preview
`digit_font`	`.byte`	150 B	3×5 pixel bitmaps for digits 0–9
`paused_font`	`.byte`	90 B	3×5 bitmaps for P-A-U-S-E-D
`gameover_font`	`.byte`	120 B	3×5 bitmaps for G-A-M-E-O-V-E-R
`columns_font`	`.byte`	105 B	3×5 bitmaps for C-O-L-U-M-N-S
`easy_font`	`.byte`	60 B	3×5 bitmaps for E-A-S-Y
`medium_font`	`.byte`	90 B	3×5 bitmaps for M-E-D-I-U-M
`hard_font`	`.byte`	60 B	3×5 bitmaps for H-A-R-D
`retry_font`	`.byte`	75 B	3×5 bitmaps for R-E-T-R-Y
`music_notes`	`.word`	1784 B	446 MIDI pitch values
`music_delays`	`.word`	1784 B	446 inter-note delay values (ms)
`music_length`	`.word`	4 B	Total note count (446)
Mutable Data			
`music_index`	`.word`	4 B	Current playback position in note arrays
`music_timer`	`.word`	4 B	Countdown to next note (ms)
`grid`	`.word`	640 B	8×20 game board, one word per cell
`col_x`	`.word`	4 B	Falling column horizontal position (0–7)
`col_y`	`.word`	4 B	Falling column vertical position (0–19)
`col_gem1/2/3`	`.word`	12 B	Color indices for top, middle, bottom gems
`col_x_prev`	`.word`	4 B	Previous x for horizontal collision detection
`match_grid`	`.word`	640 B	8×20 match flags, one word per cell
`match_found`	`.word`	4 B	Flag: any match found this pass
`game_over`	`.word`	4 B	Game state (0 = running, 1 = over)
`score`	`.word`	4 B	Current player score
`combo`	`.word`	4 B	Cascade counter for current landing
`GRAVITY_SPEED`	`.word`	4 B	Base loops per drop (30/15/5 by difficulty)
`selected_difficulty`	`.word`	4 B	Chosen difficulty (0=easy, 1=medium, 2=hard)
`score_multiplier`	`.word`	4 B	Score scaling (1×/2×/5×)
`gravity_timer`	`.word`	4 B	Loop counter toward next gravity drop

Table 1: Static and dynamic memory usage in the `.data` segment



Address	Value +0	Value +4	Value +8	Value +c	Value +10	Value +14	Value +18	Value +1c
0x10010000	0x10008000	0xffff0000	0x0	0xff0000	0xff6600	0xffff00	0xff00	0xff
0x10010020	0x9900ff	0x0	0x552200	0x552200	0x555500	0x5500	0x55	0x330055
0x10010040	0x1010101	0x10100	0x1000101	0x10101	0x1010001	0x10000	0x1000100	0x1010101
0x10010060	0x1000001	0x1010101	0x1010001	0x1010101	0x1010000	0x101	0x1010101	0x1010001
0x10010080	0x1010100	0x1000001	0x1010000	0x10101	0x1010100	0x1010000	0x1010101	0x101
0x100100a0	0x1010101	0x1010100	0x1010101	0x10000	0x1000001	0x10000	0x1010101	0x1010100
0x100100c0	0x1000101	0x1010101	0x10101	0x1010101	0x1010000	0x1010101	0x1000100	0x1000101
0x100100e0	0x10000	0x10000	0x1010001	0x10101	0x1000101	0x1010001	0x10100	0x1000101
0x10010100	0x1010101	0x10101	0x1010100	0x1010000	0x1010101	0x101	0x1000101	0x1010000
0x10010120	0x10101	0x1010001	0x10100	0x10101	0x1010101	0x10000	0x1000101	0x10101
0x10010140	0x10001	0x1010101	0x1010001	0x10100	0x1010101	0x1010001	0x10100	0x1010101
0x10010160	0x1000001	0x10001	0x1010100	0x1010101	0x10100	0x1000101	0x1010101	0x10100
0x10010180	0x1000101	0x10001	0x1010001	0x101	0x1000101	0x1010000	0x10101	0x1010001
0x100101a0	0x10001	0x1000101	0x1010101	0x10000	0x100	0x1010101	0x10101	0x1000101
0x100101c0	0x1010001	0x10101	0x100	0x1000001	0x1010000	0x1000101	0x1010001	0x10100
0x100101e0	0x1010101	0x1010001	0x10101	0x1000101	0x1010001	0x1010100	0x1010101	0x1010001
0x10010200	0x1010100	0x101	0x10101	0x1010100	0x1010101	0x1000001	0x10001	0x1010100
0x10010220	0x1000100	0x1010100	0x1000101	0x1010001	0x10101	0x1010100	0x1010000	0x10101
0x10010240	0x1000101	0x100	0x1000001	0x1000100	0x1010101	0x10100	0x1000101	0x1010101
0x10010260	0x1010000	0x100	0x1010101	0x10001	0x1000101	0x1010001	0x1010001	0x10001
0x10010280	0x100	0x1010001	0x1000101	0x1010001	0x10100	0x1010101	0x1010001	0x10101
0x100102a0	0x1000101	0x1010001	0x10100	0x1010101	0x1010001	0x1000100	0x1000100	0x1010101
0x100102c0	0x10100	0x10101	0x1010001	0x10001	0x1000101	0x1000101	0x10100	0x1000101

Figure 1: Memory Screenshot



0x10010100	0x1010101	0x10101	0x1010100	0x1010000	0x1010101	0x101	0x1000101	0x1010000
0x10010120	0x10101	0x1010001	0x10100	0x10101	0x1010101	0x10000	0x1000101	0x10101
0x10010140	0x10001	0x1010101	0x1010001	0x10100	0x1010101	0x1010001	0x10100	0x1010101
0x10010160	0x1000001	0x10001	0x1010100	0x1010101	0x10100	0x1000101	0x1010101	0x10100
0x10010180	0x1000101	0x10001	0x1010001	0x101	0x1000101	0x1010000	0x10101	0x1010001
0x100101a0	0x10001	0x1000101	0x1010101	0x10000	0x100	0x1010101	0x10101	0x1000101
0x100101c0	0x1010001	0x10101	0x100	0x1000001	0x1010000	0x1000101	0x1010001	0x10100
0x100101e0	0x1010101	0x1010001	0x10101	0x1000101	0x1010001	0x1010100	0x1010101	0x1010001
0x10010200	0x1010100	0x101	0x10101	0x1010100	0x1010101	0x1000001	0x10001	0x1010100
0x10010220	0x1000100	0x1010100	0x1000101	0x1010001	0x10101	0x1010100	0x1010000	0x10101
0x10010240	0x1000101	0x100	0x1000001	0x1000100	0x1010101	0x10100	0x1000101	0x1010101
0x10010260	0x1010000	0x100	0x1010101	0x10001	0x1000101	0x1010001	0x1010001	0x10001
0x10010280	0x100	0x1010001	0x1000101	0x1010001	0x10100	0x1010101	0x1010001	0x10101
0x100102a0	0x1000101	0x1010001	0x10100	0x1010101	0x1010001	0x1000100	0x1000100	0x1010101
0x100102c0	0x10100	0x10101	0x1010001	0x10001	0x1000101	0x1000101	0x10100	0x1000101
0x100102e0	0x1000101	0x10001	0x10101	0x1010001	0x1010100	0x101	0x1000101	0x1010000
0x10010300	0x1010101	0x100	0x1000001	0x10000	0x1000101	0x1010100	0x1000100	0x1010001
0x10010320	0x10100	0x10001	0x100	0x1	0x45	0x40	0x48	0x40
0x10010340	0x47	0x40	0x45	0x40	0x44	0x40	0x47	0x40
0x10010360	0x45	0x40	0x44	0x40	0x43	0x3e	0x47	0x3e
0x10010380	0x45	0x3e	0x43	0x3e	0x42	0x3e	0x43	0x3e
0x100103a0	0x45	0x3e	0x43	0x3e	0x45	0x40	0x48	0x40
0x100103c0	0x47	0x40	0x45	0x40	0x4a	0x40	0x44	0x40
0x100103e0	0x45	0x40	0x47	0x40	0x43	0x3e	0x47	0x3e

Figure 2: Memory Screenshot

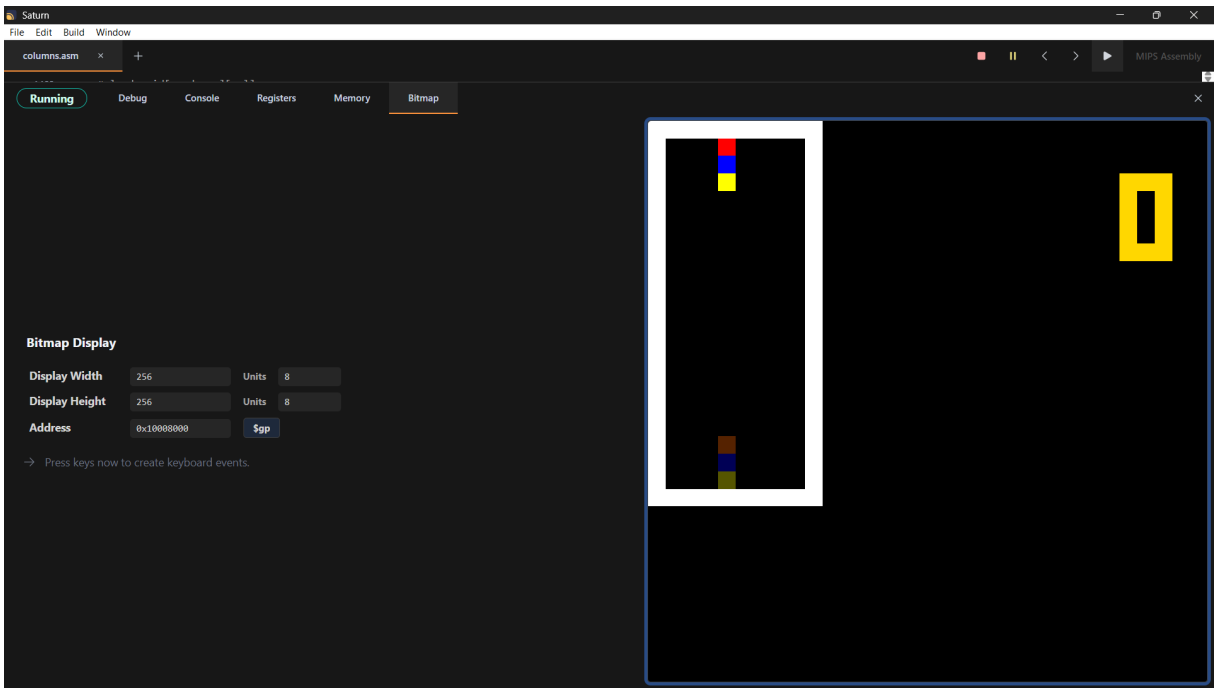


Figure 3: Milestone 1 screenshot

2 Attribution Table

Everything was completed by Erzheng Zhang, alone.

Point Calculation

When a column lands and matches are found, points are awarded as:

$$\text{points} = (\text{matched_cells} \times 50) \times 2^{(\text{combo}-1)} \times \text{difficulty_multiplier}$$

where:

- matched_cells = number of cells marked in the match grid for that pass
- combo = 1 for the first clear, increments by 1 for each subsequent cascade
- difficulty_multiplier = score scaling factor chosen at the start screen:

$$\text{difficulty_multiplier} = \begin{cases} 1 & \text{Easy} \\ 2 & \text{Medium} \\ 5 & \text{Hard} \end{cases}$$

For example, clearing 6 cells on Hard difficulty during a second cascade:

$$\text{points} = (6 \times 50) \times 2^{(2-1)} \times 5 = 300 \times 2 \times 5 = 3000$$

Gravity Acceleration

The falling speed of the column increases dynamically as the player's score grows. The effective gravity speed is computed each frame as:

$$\text{effective_speed} = \max\left(2,\ \text{base_speed} - \left\lfloor \frac{\text{score}}{2000} \right\rfloor\right)$$

where:

- effective_speed = number of game loop iterations before the column drops one row (lower = faster)
- base_speed = the starting gravity speed determined by difficulty:

$$\text{base_speed} = \begin{cases} 30 & \text{Easy} \\ 15 & \text{Medium} \\ 5 & \text{Hard} \end{cases}$$

- $\left\lfloor \frac{\text{score}}{2000} \right\rfloor$ = number of speed increments earned, one per 2000 points scored
- 2 = minimum effective speed, regardless of score, to ensure the game remains playable

For example, on Medium difficulty with a score of 8000:

$$\text{effective_speed} = \max(2,\ 15 - 4) = 11$$

The column falls noticeably faster than at the start, but the game loop still runs at 60fps (16ms per frame via `sleep`).

Difficulty Selection and Level Menu

Three difficulty levels are available, selected from the start screen using `w/s` to navigate and Enter to confirm. The selected option is highlighted in white while unselected options are dimmed. Difficulty affects two parameters simultaneously:

- `GRAVITY_SPEED`: base number of game loop iterations before the column drops one row
- `score_multiplier`: flat multiplier applied to every point calculation

Difficulty	Base Gravity Speed	Score Multiplier
Easy	30	1×
Medium	15	2×
Hard	5	5×

Gravity and Gravity Acceleration

The column automatically falls one row every `GRAVITY_SPEED` game loop iterations. A `gravity_timer` counter increments each loop and resets on a drop. On top of this, the effective gravity speed increases as the player scores more points (see the Gravity Acceleration formula above), so the game naturally becomes harder the longer it is played.

The player can also trigger an instant hard drop with `s`, which scans downward from the current position until hitting the floor or an occupied cell, then locks the column immediately.

Game Over and Retry

Game over is triggered when a new column would spawn at row 0 but the landing position was also row 0, meaning the stack has reached the top. At this point the game draws a `GAME OVER` overlay followed by a `RETRY` prompt. The game then waits for:

- `r`: resets all state (grid, score, combo, timers) and returns to the start screen
- `q`: exits the program immediately

The full reset clears the grid array and all mutable state before reinitializing, so no data from the previous run persists.

Pause

Pressing `p` during gameplay draws the `PAUSED` overlay and enters a busy-wait loop that polls the keyboard. The game loop is fully suspended: no gravity, music advancement, or drawing occurs. Pressing `p` again exits the loop and resumes exactly where it left off.

Ghost Piece

The ghost piece shows a dimmed preview of where the current column will land. It is computed each frame by simulating the same downward scan used by the hard drop, without modifying `col_y`. The ghost is drawn using a separate `ghost_color_table` with darkened versions of each gem color. If the ghost position equals the current column position (i.e. the column is already at the bottom), the ghost is skipped to avoid drawing over the live piece.

Score Display

The current score is rendered on the right side of the display each frame as up to 5 digits in a gold (`0xffd700`) 3×5 pixel font. Digits are extracted right-to-left via repeated division by 10, stored in a stack buffer, then drawn left-to-right. Leading digit slots are padded with -1 and skipped during rendering so no leading zeros appear.

Music

Background music plays throughout the game using MIPS `syscall 31` (asynchronous MIDI output), which is non-blocking and does not interrupt the game loop. The piece is a 446-note marimba arrangement stored as two parallel arrays: `music_notes` (MIDI pitch values) and `music_delays` (milliseconds until the next note). Each call to `update_music` decrements a timer by 16ms; when it expires, the next note is fired and the timer is reloaded from `music_delays`. The sequence loops continuously. Music also plays on the start screen during difficulty selection.